



**FEDERAL URDU UNIVERSITY OF ARTS, SCIENCE AND TECHNOLOGY  
ISLAMABAD**

**DEPARTMENT OF COMPUTER SCIENCE**

## **Operating Systems**

### **Project Documentation**

#### **Intelligent Traffic Management and Deadlock Prevention System Using Operating System Concepts**

**Student Name(s):**

1. Taimour Mushtaq
2. Hafiz Muhammad Abdullah Idrees
3. Abdur Rahman
4. Zahid Ali Akbar

**Semester:** 4<sup>th</sup>

**Section:** A

**Submitted To:** Ms. Kinza Naseer

**Date Submitted:** June 03, 2026

Signed: \_\_\_\_\_

## **Table of Contents**

|                                             |    |
|---------------------------------------------|----|
| Project Title.....                          | 5  |
| Proposed Technology Stack.....              | 5  |
| 1. Introduction .....                       | 5  |
| 2. Problem Statement .....                  | 5  |
| 3. Objectives .....                         | 6  |
| 4. Scope of the Project .....               | 6  |
| 5. Proposed System Overview .....           | 7  |
| 6. Operating System Concepts Used .....     | 7  |
| 6.1 Process Creation (fork()) .....         | 7  |
| 6.2 Process Waiting (wait()) .....          | 7  |
| 6.3 Sleep Function (sleep()) .....          | 7  |
| 7. CPU Scheduling Algorithms .....          | 7  |
| 7.1 First Come First Serve (FCFS).....      | 7  |
| 7.2 Shortest Job First (SJF) .....          | 8  |
| 7.3 Round Robin (RR) .....                  | 8  |
| 7.4 Multilevel Queue Scheduling (MQS).....  | 8  |
| 8. Synchronization Mechanisms.....          | 8  |
| 8.1 Mutex .....                             | 8  |
| 8.2 Semaphores .....                        | 8  |
| 9. Banker's Algorithm .....                 | 9  |
| 10. Deadlock Management.....                | 9  |
| 11. Signals and IPC .....                   | 9  |
| 11.1 Signals .....                          | 9  |
| 11.2 Inter-Process Communication (IPC)..... | 9  |
| 12. System Modules .....                    | 10 |
| 13. Functional Requirements .....           | 10 |
| 14. Non-Functional Requirements .....       | 10 |
| 15. Expected Output .....                   | 11 |
| 16. Feasibility Analysis.....               | 11 |
| Technical Feasibility .....                 | 11 |
| Operational Feasibility .....               | 11 |
| 17. Development Methodology.....            | 11 |
| 18. Work Timeline.....                      | 11 |

|                                    |    |
|------------------------------------|----|
| 19. Advantages of the System ..... | 12 |
| 20. Future Enhancements .....      | 12 |
| 21. Conclusion.....                | 12 |
| 22. Final Project Visuals .....    | 13 |
| 23. References .....               | 20 |

## **Table of Figures**

|                                          |    |
|------------------------------------------|----|
| Figure 1 Main Menu .....                 | 13 |
| Figure 2 FCFS.....                       | 13 |
| Figure 3 SJF .....                       | 14 |
| Figure 4 RR.....                         | 14 |
| Figure 5 Gantt Chart & Output of RR..... | 15 |
| Figure 6 MQS .....                       | 15 |
| Figure 7 Mutex + Semaphore .....         | 16 |
| Figure 8 Race Condition Results .....    | 16 |
| Figure 9 Banker's Algorithm.....         | 17 |
| Figure 10 Results of Banker's Algo ..... | 17 |
| Figure 11 Deadlock Simulation .....      | 17 |
| Figure 12 IPC Demo 1 .....               | 18 |
| Figure 13 IPC Demo 2 .....               | 18 |
| Figure 14 IPC Demo 3 .....               | 19 |
| Figure 15 Vehicle Table .....            | 19 |
| Figure 16 End of Project.....            | 20 |

## Project Title

# Intelligent Traffic Management and Deadlock Prevention System Using Operating System Concepts

## Proposed Technology Stack

| Component            | Technology                           |
|----------------------|--------------------------------------|
| Programming Language | C                                    |
| Operating System     | Ubuntu Linux                         |
| Virtual Environment  | VMware                               |
| Compiler             | GCC                                  |
| Thread Library       | POSIX Threads (pthread)              |
| Synchronization      | Mutex & Semaphores                   |
| IPC Mechanisms       | Pipes, Shared Memory, Message Queues |
| Terminal Interface   | Linux Terminal / ncurses (optional)  |

## 1. Introduction

Modern traffic systems face challenges such as congestion, inefficient scheduling of vehicles, deadlocks at intersections, and improper resource allocation. These issues resemble problems handled by operating systems, including process scheduling, synchronization, deadlock prevention, inter-process communication, and resource management.

This project proposes the development of an Intelligent Traffic Management and Deadlock Prevention System that simulates real-world traffic flow while implementing major operating system concepts in Ubuntu Linux.

The project will demonstrate how operating system algorithms and synchronization mechanisms can efficiently manage concurrent processes and shared resources.

The system will simulate vehicles as processes or threads, traffic signals as synchronization controllers, and intersections as shared critical resources. Different CPU scheduling algorithms will be used to control traffic flow and reduce waiting time.

## 2. Problem Statement

Managing multiple vehicles simultaneously at road intersections can create conflicts similar to process synchronization issues in operating systems. Without proper scheduling and synchronization mechanisms, traffic congestion and deadlocks may occur.

The project aims to solve these problems by implementing operating system concepts such as:

- CPU Scheduling

- Process Creation
- Synchronization
- Deadlock Prevention
- Resource Allocation
- Inter-Process Communication

within a simulated traffic management environment.

### 3. Objectives

The major objectives of the project are:

1. To simulate a real-time traffic management system using operating system concepts.
2. To implement process creation and management using `fork()`.
3. To demonstrate process synchronization using mutexes and semaphores.
4. To implement CPU scheduling algorithms:
  - First Come First Serve (FCFS)
  - Shortest Job First (SJF)
  - Round Robin (RR)
  - Multilevel Queue Scheduling (MQS)
5. To implement Banker's Algorithm for safe resource allocation.
6. To simulate deadlock situations and demonstrate prevention techniques.
7. To implement Inter-Process Communication (IPC).
8. To use Linux signals for process control and communication.
9. To provide practical understanding of operating system concepts.

### 4. Scope of the Project

The project scope includes:

- Simulation of traffic signals and vehicle movement.
- Scheduling of vehicle processes.
- Resource allocation for traffic lanes and intersections.
- Prevention and detection of deadlocks.
- Communication between traffic controller and vehicles.
- Synchronization of shared resources.

The system will run entirely on Ubuntu Linux using terminal-based interaction.

## 5. Proposed System Overview

The system consists of:

- Vehicle Processes
- Traffic Controller
- Scheduling Unit
- Resource Allocation Manager
- Deadlock Detection Module
- IPC Communication Module

Each vehicle entering the traffic system will behave like a process competing for shared resources such as road lanes and intersections.

## 6. Operating System Concepts Used

### 6.1 Process Creation (fork())

Vehicle processes will be generated dynamically using:

`fork()`

Each child process will represent a vehicle entering the system.

### 6.2 Process Waiting (wait())

Parent processes will wait for child vehicle processes to complete execution using:

`wait()`

### 6.3 Sleep Function (sleep())

The `sleep()` function will simulate:

- signal delays
- vehicle waiting time
- traffic light duration

Example:

`sleep(2);`

## 7. CPU Scheduling Algorithms

The project will implement multiple scheduling algorithms.

### 7.1 First Come First Serve (FCFS)

Vehicles are processed in order of arrival.

*Purpose:*

- Simple traffic handling mechanism.

## 7.2 Shortest Job First (SJF)

Vehicles requiring less processing time receive priority.

*Purpose:*

- Reduce average waiting time.

## 7.3 Round Robin (RR)

Each lane receives a fixed time quantum.

*Purpose:*

- Fair allocation of road access.

## 7.4 Multilevel Queue Scheduling (MQS)

Different vehicle categories:

- Emergency Vehicles
- Public Transport
- Regular Vehicles

Each queue will have different priority levels.

# 8. Synchronization Mechanisms

## 8.1 Mutex

Mutex locks will protect critical sections such as:

- shared traffic data
- road counters
- signal states

*Purpose:*

Prevent race conditions.

## 8.2 Semaphores

Semaphores will control:

- lane access
- intersection occupancy
- limited parking spaces

*Purpose:*

Allow safe concurrent access.

## 9. Banker's Algorithm

Banker's Algorithm will be implemented to:

- allocate road resources safely
- avoid unsafe traffic states
- prevent system deadlocks

The algorithm will check whether granting access to a lane keeps the system in a safe state.

## 10. Deadlock Management

The project will simulate traffic deadlock situations where:

- vehicles block each other at intersections
- no vehicle can proceed further

The system will:

- detect deadlocks
- prevent unsafe allocations
- recover from blocked states

## 11. Signals and IPC

### 11.1 Signals

Linux signals will be used for:

- emergency vehicle handling
- process interruption
- pausing/resuming traffic flow

Example signals:

- SIGSTOP
- SIGCONT
- SIGINT

### 11.2 Inter-Process Communication (IPC)

IPC techniques include:

- Pipes

- Shared Memory
- Message Queues

These will enable communication between:

- traffic controller
- vehicle processes
- scheduler

## 12. System Modules

| Module                         | Description                     |
|--------------------------------|---------------------------------|
| <b>Vehicle Generator</b>       | Creates vehicle processes       |
| <b>Traffic Scheduler</b>       | Applies scheduling algorithms   |
| <b>Synchronization Manager</b> | Handles mutexes/semaphores      |
| <b>Deadlock Detector</b>       | Detects circular waits          |
| <b>Banker Module</b>           | Checks safe resource allocation |
| <b>IPC Module</b>              | Enables process communication   |
| <b>Signal Handler</b>          | Handles emergency controls      |

## 13. Functional Requirements

The system should:

- create multiple processes
- schedule traffic efficiently
- avoid resource conflicts
- detect deadlocks
- communicate using IPC
- handle emergency interrupts

## 14. Non-Functional Requirements

The system should:

- run on Ubuntu Linux
- execute efficiently
- maintain synchronization
- provide understandable terminal output

- support multiple concurrent processes

## 15. Expected Output

The project will:

- visually simulate traffic flow in terminal
- show scheduling execution
- display synchronization operations
- detect and prevent deadlocks
- demonstrate Banker's Algorithm decisions
- show IPC communication between processes

## 16. Feasibility Analysis

### Technical Feasibility

The project is technically feasible because:

- Ubuntu supports all required OS features
- GCC compiler supports pthreads and IPC
- Linux provides native process management tools

### Operational Feasibility

The project can be implemented within the given timeline because:

- modular architecture
- terminal-based implementation
- available Linux libraries

## 17. Development Methodology

The project will follow:

*Incremental Development Methodology*

Each module will be developed and tested separately.

## 18. Work Timeline

| Days      | Tasks                                    |
|-----------|------------------------------------------|
| Day 1-2   | Project setup and planning               |
| Day 3-4   | Process creation module                  |
| Day 5-6   | Scheduling algorithms                    |
| Day 7-8   | Mutex and semaphore implementation       |
| Day 9-10  | Banker's Algorithm and deadlock handling |
| Day 11-12 | IPC and signal handling                  |
| Day 13    | Testing and debugging                    |
| Day 14    | Documentation and presentation           |

## 19. Advantages of the System

- Demonstrates practical OS concepts
- Simulates real-world traffic systems
- Provides understanding of synchronization
- Demonstrates resource management
- Enhances Linux programming skills

## 20. Future Enhancements

Future improvements may include:

- graphical interface
- AI-based traffic optimization
- network-based distributed traffic system
- real-time sensor integration
- database connectivity

## 21. Conclusion

The proposed Intelligent Traffic Management and Deadlock Prevention System provide a practical implementation of major operating system concepts in a real-world scenario. The project integrates process management, CPU scheduling, synchronization, deadlock handling, IPC, and signal processing into a unified Linux-based simulation.

The project will enhance understanding of operating system fundamentals while providing hands-on experience with Ubuntu system programming.

## 22. Final Project Visuals

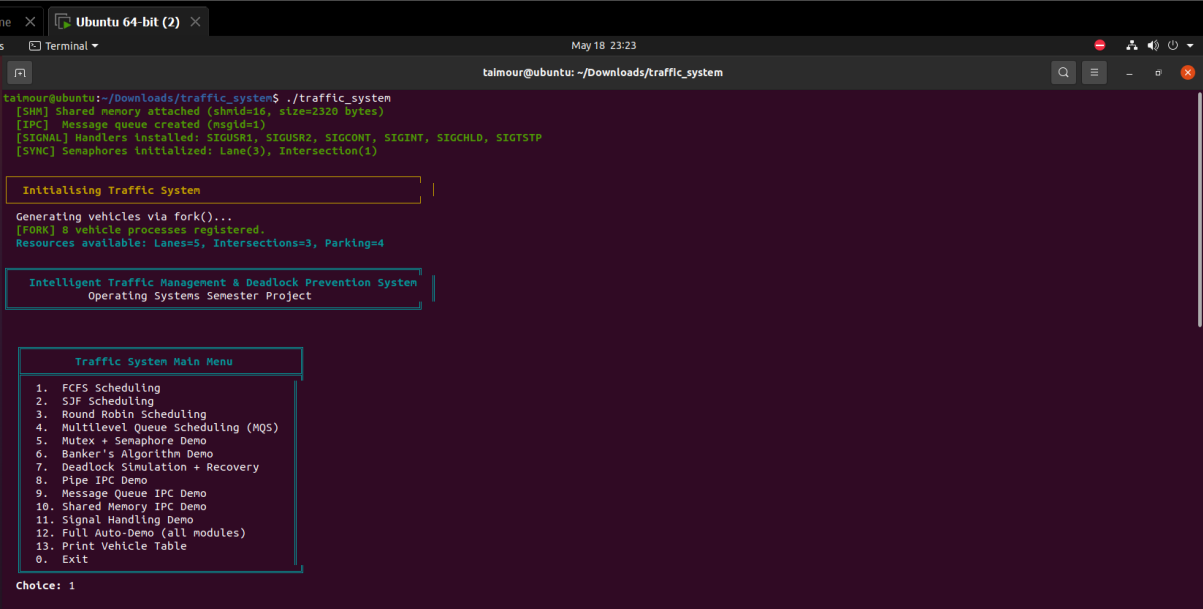


Figure 1 Main Menu

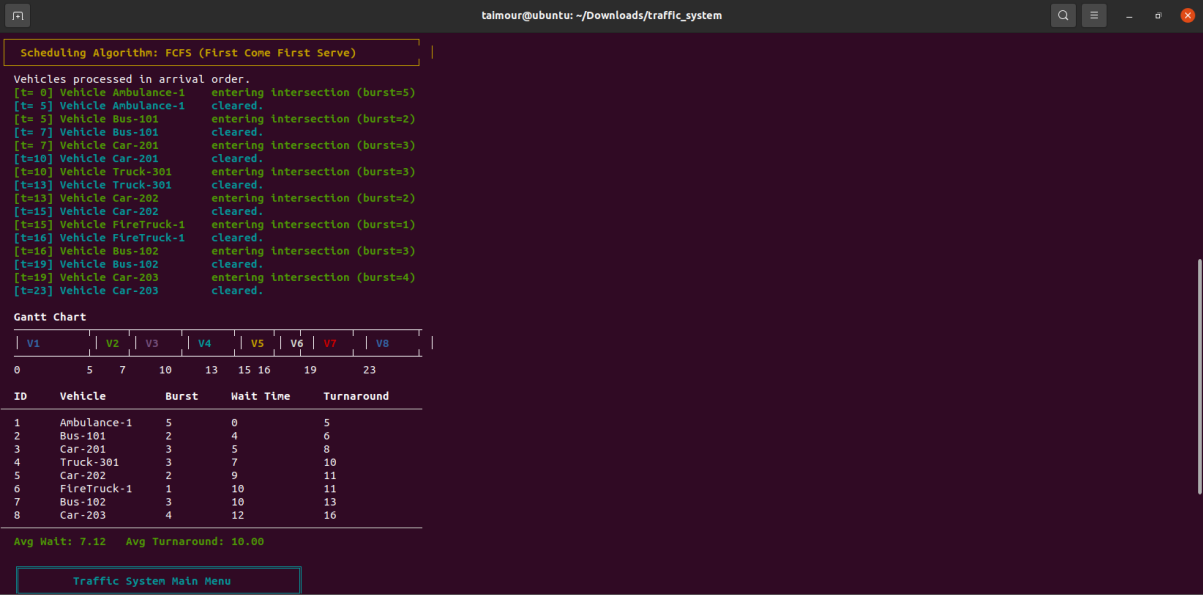


Figure 2 FCFS

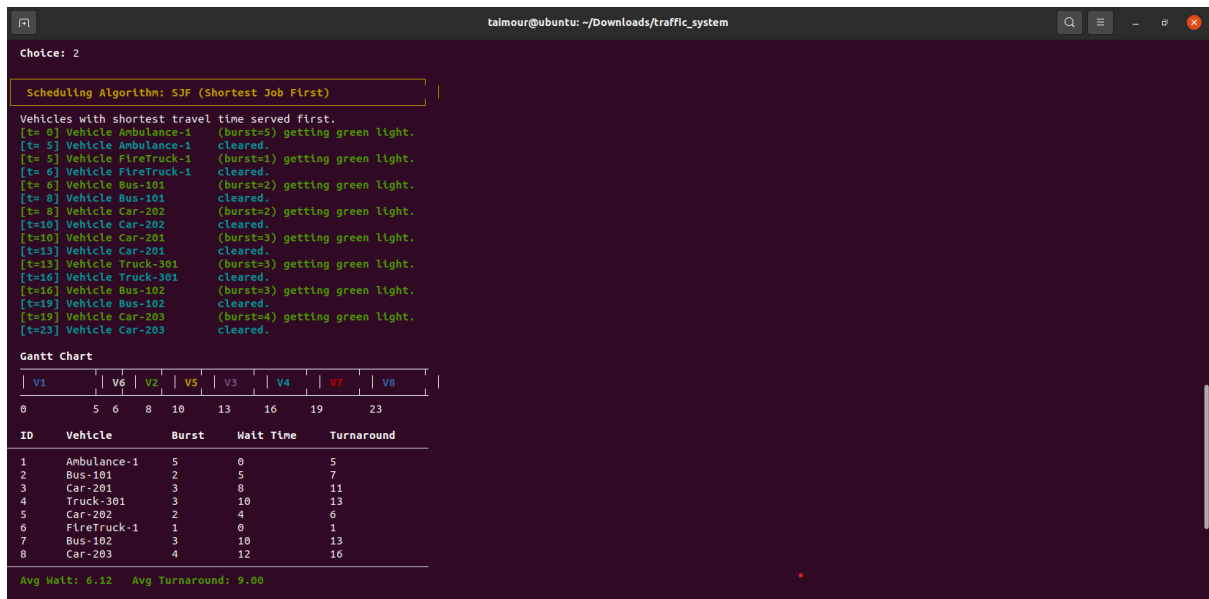


Figure 3 SJF

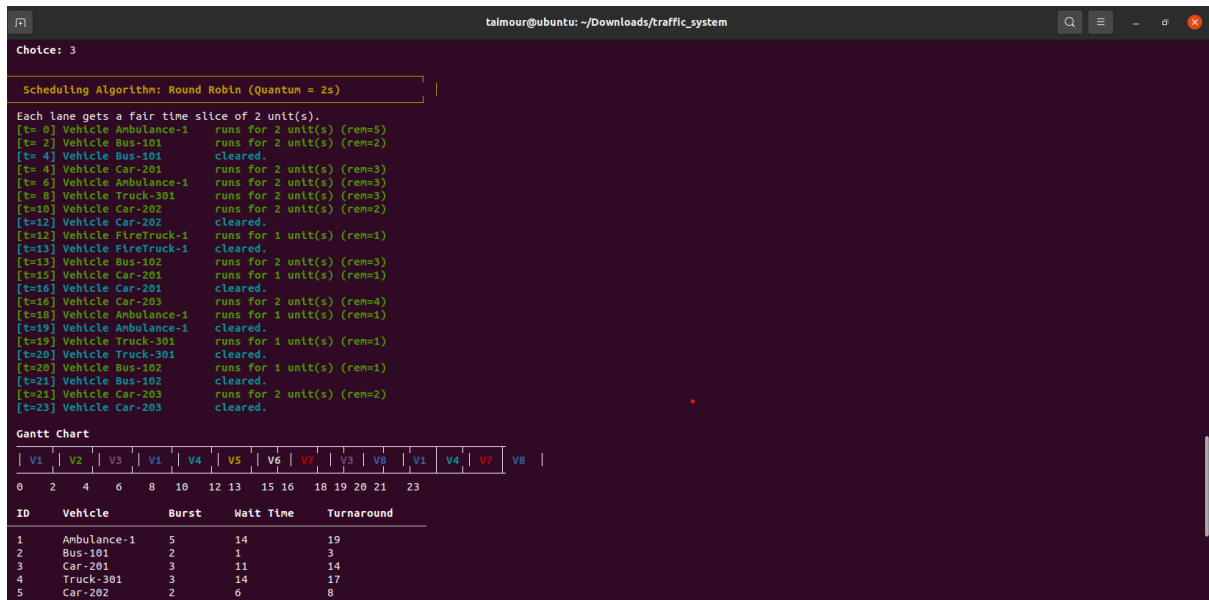


Figure 4 RR



Figure 5 Gantt Chart & Output of RR

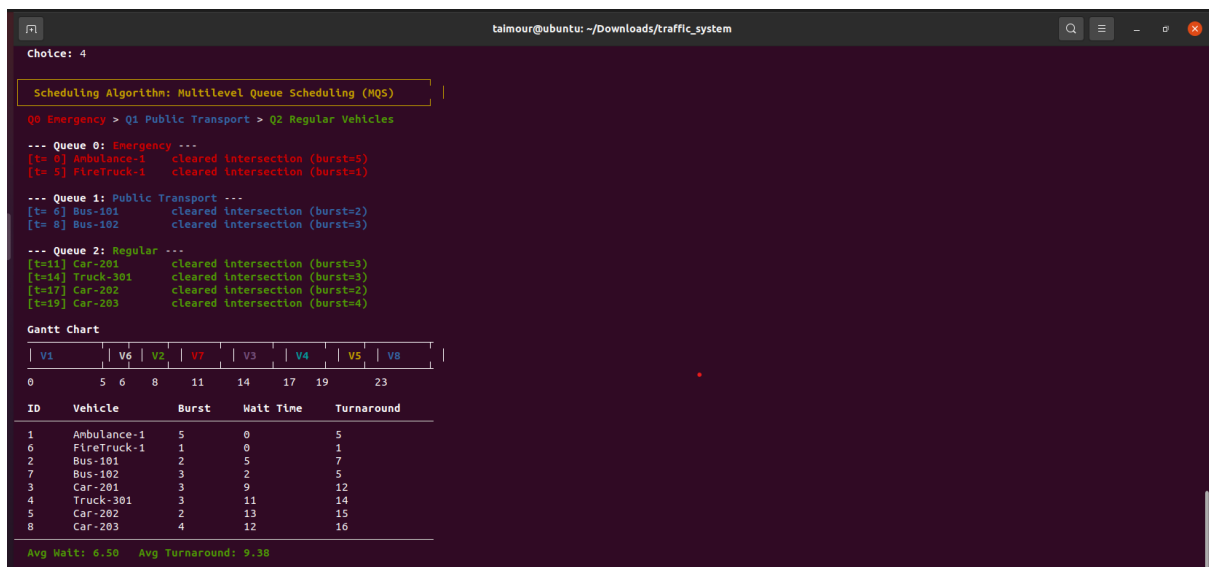


Figure 6 MQS

```
talmour@ubuntu: ~/Downloads/traffic_system
Choice: 5
Synchronization Demo: Mutex + Semaphores
8 vehicle threads compete for lane (cap=3) and intersection (cap=1).

[SYNC] Vehicle 1 waiting for lane semaphore...
[SYNC] Vehicle 1 acquired lane semaphore.
[SYNC] Vehicle 2 waiting for lane semaphore...
[SYNC] Vehicle 2 acquired lane semaphore.
[SYNC] Vehicle 3 waiting for lane semaphore...
[SYNC] Vehicle 3 acquired lane semaphore.
[SYNC] Vehicle 4 waiting for lane semaphore...
[MUTEX] Vehicle 1 waiting for intersection mutex...
[MUTEX] Vehicle 1 locked intersection (critical section).
[THREAD] Vehicle 1 passing through intersection.
[SYNC] Vehicle 5 waiting for lane semaphore...
[SYNC] Vehicle 6 waiting for lane semaphore...
[SYNC] Vehicle 6 waiting for intersection mutex...
[MUTEX] Vehicle 3 waiting for intersection mutex...
[SYNC] Vehicle 7 waiting for lane semaphore...
[SYNC] Vehicle 8 waiting for lane semaphore...
[MUTEX] Vehicle 1 unlocked intersection.
[SYNC] Vehicle 1 released lane semaphore.
[MUTEX] Vehicle 2 locked intersection (critical section).
[THREAD] Vehicle 2 passing through intersection.
[SYNC] Vehicle 4 acquired lane semaphore.
[MUTEX] Vehicle 4 waiting for intersection mutex...
[MUTEX] Vehicle 2 unlocked intersection.
[SYNC] Vehicle 2 released lane semaphore.
[MUTEX] Vehicle 3 locked intersection (critical section).
[THREAD] Vehicle 3 passing through intersection.
[SYNC] Vehicle 5 acquired lane semaphore.
[MUTEX] Vehicle 5 waiting for intersection mutex...
[MUTEX] Vehicle 3 unlocked intersection.
[SYNC] Vehicle 3 released lane semaphore.
[SYNC] Vehicle 6 acquired lane semaphore.
[MUTEX] Vehicle 4 locked intersection (critical section).
[THREAD] Vehicle 4 passing through intersection.
[MUTEX] Vehicle 6 waiting for intersection mutex...
[MUTEX] Vehicle 4 unlocked intersection.
[SYNC] Vehicle 4 released lane semaphore.
```

Figure 7 Mutex + Semaphore

```
talmour@ubuntu: ~/Downloads/traffic_system
[SYNC] Vehicle 2 released lane semaphore.
[MUTEX] Vehicle 3 locked intersection (critical section).
[THREAD] Vehicle 3 passing through intersection.
[SYNC] Vehicle 5 acquired lane semaphore.
[MUTEX] Vehicle 5 waiting for intersection mutex...
[MUTEX] Vehicle 3 unlocked intersection.
[SYNC] Vehicle 3 released lane semaphore.
[SYNC] Vehicle 6 acquired lane semaphore.
[MUTEX] Vehicle 4 locked intersection (critical section).
[THREAD] Vehicle 4 passing through intersection.
[MUTEX] Vehicle 6 waiting for intersection mutex...
[MUTEX] Vehicle 4 unlocked intersection.
[SYNC] Vehicle 4 released lane semaphore.
[SYNC] Vehicle 7 acquired lane semaphore.
[MUTEX] Vehicle 5 locked intersection (critical section).
[THREAD] Vehicle 5 passing through intersection.
[MUTEX] Vehicle 7 waiting for intersection mutex...
[MUTEX] Vehicle 5 unlocked intersection.
[SYNC] Vehicle 5 released lane semaphore.
[MUTEX] Vehicle 6 locked intersection (critical section).
[THREAD] Vehicle 6 passing through intersection.
[SYNC] Vehicle 8 acquired lane semaphore.
[MUTEX] Vehicle 7 waiting for intersection mutex...
[MUTEX] Vehicle 6 unlocked intersection.
[SYNC] Vehicle 6 released lane semaphore.
[MUTEX] Vehicle 7 locked intersection (critical section).
[THREAD] Vehicle 7 passing through intersection.
[MUTEX] Vehicle 7 unlocked intersection.
[SYNC] Vehicle 7 released lane semaphore.
[MUTEX] Vehicle 8 locked intersection (critical section).
[THREAD] Vehicle 8 passing through intersection.
[MUTEX] Vehicle 8 unlocked intersection.
[SYNC] Vehicle 8 released lane semaphore.
[SYNC] All vehicles passed. No race conditions detected.
```

Figure 8 Race Condition Results

```
Choice: 6

Banker's Algorithm - Safe Resource Allocation

[BANKER] Initialized. Resources: Lanes=5, Intersections=3, Parking=4

ID  Vehicle      Max Need (L/I/P)  Allocated (L/I/P)  Need (L/I/P)
1   Ambulance-1   [1 3 1]           [0 0 0]             [1 3 1]
2   Bus-101       [1 3 3]           [0 0 0]             [1 3 3]
3   Car-201       [1 2 1]           [0 0 0]             [1 2 1]
4   Truck-301     [3 3 3]           [0 0 0]             [3 3 3]
5   Car-202       [3 1 2]           [0 0 0]             [3 1 2]
6   FireTruck-1   [2 1 1]           [0 0 0]             [2 1 1]
7   Bus-102       [3 1 1]           [0 0 0]             [3 1 1]
8   Car-203       [3 3 2]           [0 0 0]             [3 3 2]

Available Resources → Lanes: 5 Intersections: 3 Parking: 4

Running safety check on initial state...
[BANKER] System is in a SAFE state.
Safe sequence: V1 → V2 → V3 → V4 → V5 → V6 → V7 → V8

Simulating resource requests...
[BANKER] Vehicle 1 requests [1, 0, 1]
[BANKER] System is in a SAFE state.
Safe sequence: V1 → V2 → V3 → V4 → V5 → V6 → V7 → V8
[BANKER] Request GRANTED for Vehicle 1.
[BANKER] Vehicle 2 requests [0, 1, 0]
[BANKER] System is in a SAFE state.
Safe sequence: V2 → V3 → V4 → V5 → V6 → V7 → V8 → V1
[BANKER] Request GRANTED for Vehicle 2.
[BANKER] Vehicle 3 requests [2, 1, 1]
[BANKER] Error: request exceeds declared maximum need.
```

Figure 9 Banker's Algorithm

```
State after allocations:

ID  Vehicle      Max Need (L/I/P)  Allocated (L/I/P)  Need (L/I/P)
1   Ambulance-1   [1 3 1]           [1 0 1]             [0 3 0]
2   Bus-101       [1 3 3]           [0 1 0]             [1 2 3]
3   Car-201       [1 2 1]           [0 0 0]             [1 2 1]
4   Truck-301     [3 3 3]           [0 0 0]             [3 3 3]
5   Car-202       [3 1 2]           [0 0 0]             [3 1 2]
6   FireTruck-1   [2 1 1]           [0 0 0]             [2 1 1]
7   Bus-102       [3 1 1]           [0 0 0]             [3 1 1]
8   Car-203       [3 3 2]           [0 0 0]             [3 3 2]

Available Resources → Lanes: 4 Intersections: 2 Parking: 3
[BANKER] Vehicle 1 released resources.
[BANKER] Vehicle 2 released resources.
[BANKER] Resources released. Final safe state confirmed.

ID  Vehicle      Max Need (L/I/P)  Allocated (L/I/P)  Need (L/I/P)
1   Ambulance-1   [1 3 1]           [0 0 0]             [1 3 1]
2   Bus-101       [1 3 3]           [0 0 0]             [1 3 3]
3   Car-201       [1 2 1]           [0 0 0]             [1 2 1]
4   Truck-301     [3 3 3]           [0 0 0]             [3 3 3]
5   Car-202       [3 1 2]           [0 0 0]             [3 1 2]
6   FireTruck-1   [2 1 1]           [0 0 0]             [2 1 1]
7   Bus-102       [3 1 1]           [0 0 0]             [3 1 1]
8   Car-203       [3 3 2]           [0 0 0]             [3 3 2]

Available Resources → Lanes: 5 Intersections: 3 Parking: 4
```

Figure 10 Results of Banker's Algo

```
Choice: 7

Deadlock Simulation - Circular Wait at Intersection

Two vehicles will hold one resource each and request the other.

[Vehicle-A PID=2174] Acquired Intersection resource.
[PARENT] Vehicle-A PID=2174 Vehicle-B PID=2175
[Vehicle-A PID=2174] Requesting Parking resource...
[Vehicle-B PID=2175] Acquired Parking resource.
[Vehicle-B PID=2175] Requesting Intersection resource...
[Vehicle-B PID=2175] Waiting for Intersection... (blocked)
[Vehicle-A PID=2174] Waiting for Parking... (blocked)

[WATCHDOG] Scanning resource allocation graph for cycles...
[WATCHDOG] Circular wait detected!
Vehicle-A holds Intersection, waiting for Parking.
Vehicle-B holds Parking, waiting for Intersection.
[WATCHDOG] DEADLOCK CONFIRMED.

[RECOVERY] Killing Vehicle-B (PID 2175) to break deadlock.
[PARENT] PID 2175 terminated by signal 15.
[IPC] Message queue removed.
[IPC] Shared memory removed.
[PARENT] PID 2174 exited normally.

[RECOVERY] Deadlock resolved. Remaining vehicle may proceed.
```

Figure 11 Deadlock Simulation

```
talmour@ubuntu: ~/Downloads/traffic_system

3. Round Robin Scheduling
4. Multilevel Queue Scheduling (MQS)
5. Mutex + Semaphore Demo
6. Banker's Algorithm Demo
7. Deadlock Simulation + Recovery
8. Pipe IPC Demo
9. Message Queue IPC Demo
10. Shared Memory IPC Demo
11. Signal Handling Demo
12. Full Auto-Demo (all modules)
13. Print Vehicle Table
0. Exit

Choice: 8

IPC Demo 1: Pipe (Traffic Controller → Vehicle)

[CONTROLLER] Sending via pipe: "GREEN LIGHT - Intersection clear, proceed!"
[VEHICLE PID=2313] Signal received via pipe: "GREEN LIGHT - Intersection clear, proceed!"
[IPC] Message queue removed.
[IPC] Shared Memory removed.

Traffic System Main Menu

1. FCFS Scheduling
2. SJF Scheduling
3. Round Robin Scheduling
4. Multilevel Queue Scheduling (MQS)
5. Mutex + Semaphore Demo
6. Banker's Algorithm Demo
7. Deadlock Simulation + Recovery
8. Pipe IPC Demo
9. Message Queue IPC Demo
10. Shared Memory IPC Demo
11. Signal Handling Demo
12. Full Auto-Demo (all modules)
13. Print Vehicle Table
0. Exit
```

Figure 12 IPC Demo 1

```
talmour@ubuntu: ~/Downloads/traffic_system

5. Mutex + Semaphore Demo
6. Banker's Algorithm Demo
7. Deadlock Simulation + Recovery
8. Pipe IPC Demo
9. Message Queue IPC Demo
10. Shared Memory IPC Demo
11. Signal Handling Demo
12. Full Auto-Demo (all modules)
13. Print Vehicle Table
0. Exit

Choice: 9

IPC Demo 2: Message Queue (Priority Messages)

msgsnd: Invalid argument
msgsnd: Invalid argument
msgsnd: Invalid argument
msgsnd: Invalid argument

[IPC] Receiving all messages from queue:

Traffic System Main Menu

1. FCFS Scheduling
2. SJF Scheduling
3. Round Robin Scheduling
4. Multilevel Queue Scheduling (MQS)
5. Mutex + Semaphore Demo
6. Banker's Algorithm Demo
7. Deadlock Simulation + Recovery
8. Pipe IPC Demo
9. Message Queue IPC Demo
10. Shared Memory IPC Demo
11. Signal Handling Demo
12. Full Auto-Demo (all modules)
13. Print Vehicle Table
0. Exit

Choice: █
```

Figure 13 IPC Demo 2

```
talmour@ubuntu: ~/Downloads/traffic_system

5. Mutex + Semaphore Demo
6. Banker's Algorithm Demo
7. Deadlock Simulation + Recovery
8. Pipe IPC Demo
9. Message Queue IPC Demo
10. Shared Memory IPC Demo
11. Signal Handling Demo
12. Full Auto-Demo (all modules)
13. Print Vehicle Table
0. Exit

Choice: 10

IPC Demo 3: Shared Memory (Traffic Controller ↔ Vehicles)

[CONTROLLER] Writing emergency flag to shared memory.
[VEHICLE PID=2341] Read emergency flag from shared mem: 1
[VEHICLE PID=2341] Pulling over for emergency vehicle!
[IPC] Message queue removed.
[IPC] Shared memory removed.
[CONTROLLER] Emergency flag cleared.

Traffic System Main Menu
1. FCFS Scheduling
2. SJF Scheduling
3. Round Robin Scheduling
4. Multilevel Queue Scheduling (MQS)
5. Mutex + Semaphore Demo
6. Banker's Algorithm Demo
7. Deadlock Simulation + Recovery
8. Pipe IPC Demo
9. Message Queue IPC Demo
10. Shared Memory IPC Demo
11. Signal Handling Demo
12. Full Auto-Demo (all modules)
13. Print Vehicle Table
0. Exit

Choice: 
```

Figure 14 IPC Demo 3

```
talmour@ubuntu: ~/Downloads/traffic_system

7. Deadlock Simulation + Recovery
8. Pipe IPC Demo
9. Message Queue IPC Demo
10. Shared Memory IPC Demo
11. Signal Handling Demo
12. Full Auto-Demo (all modules)
13. Print Vehicle Table
0. Exit

Choice: 13

ID  Vehicle      Queue      Arrival  Burst  State  PID
1   Ambulance-1   Emergency  0        5      WAITING -1
2   Bus-101       Public    1        2      WAITING -1
3   Car-201       Regular   2        3      WAITING -1
4   Truck-301     Regular   3        3      WAITING -1
5   Car-202       Regular   4        2      WAITING -1
6   FireTruck-1   Emergency  5        1      WAITING -1
7   Bus-102       Public    6        3      WAITING -1
8   Car-203       Regular   7        4      WAITING -1

Traffic System Main Menu
1. FCFS Scheduling
2. SJF Scheduling
3. Round Robin Scheduling
4. Multilevel Queue Scheduling (MQS)
5. Mutex + Semaphore Demo
6. Banker's Algorithm Demo
7. Deadlock Simulation + Recovery
8. Pipe IPC Demo
9. Message Queue IPC Demo
10. Shared Memory IPC Demo
11. Signal Handling Demo
12. Full Auto-Demo (all modules)
13. Print Vehicle Table
0. Exit

Choice: 
```

Figure 15 Vehicle Table

```
talmour@ubuntu: ~/Downloads/traffic_system
13. Print Vehicle Table
0. Exit
Choice: 13
ID  Vehicle      Queue      Arrival  Burst  State  PID
1   Ambulance-1   Emergency  0        5      WAITING -1
2   Bus-101       Public     1        2      WAITING -1
3   Car-201       Regular    2        3      WAITING -1
4   Truck-301     Regular    3        3      WAITING -1
5   Car-202       Regular    4        2      WAITING -1
6   FireTruck-1   Emergency  5        1      WAITING -1
7   Bus-102       Public     6        3      WAITING -1
8   Car-203       Regular    7        4      WAITING -1

Traffic System Main Menu
1. FCFS Scheduling
2. SJF Scheduling
3. Round Robin Scheduling
4. Multilevel Queue Scheduling (MQS)
5. Mutex + Semaphore Demo
6. Banker's Algorithn Demo
7. Deadlock Simulation + Recovery
8. Pipe IPC Demo
9. Message Queue IPC Demo
10. Shared Memory IPC Demo
11. Signal Handling Demo
12. Full Auto-Demo (all modules)
13. Print Vehicle Table
0. Exit
Choice: 0
Traffic system shutting down. Goodbye!
[IPC] Message queue removed.
[IPC] Shared memory removed.
talmour@ubuntu: ~/Downloads/traffic_system$
```

Figure 16 End of Project

## 23. References

1. Operating System Concepts Abraham Silberschatz
2. Advanced Programming in the UNIX Environment W. Richard Stevens
3. Linux Manual Pages
4. GNU GCC Documentation
5. POSIX Thread Programming Guide
6. Ubuntu Linux Documentation